MEMORIA TÉCNICA Y METODOLOGÍA DE EJECUCIÓN

Anexo IV: Registro Forense de Operaciones y Procedimiento en Hashcat (CAI 1)

Asignatura: Seguridad en Sistemas de Información e Internet | **Equipo Auditor:** Security Team 2 (ST-2)
Autores: Raimundo García Pérez, Konstantinos Peridis, Luis Alarcón Conradi
Entorno de Pruebas: Kali Linux (Virtualizado) | **Documento de Referencia:** ST-2-CAI1A

---

1. Entorno de Auditoría y Configuración del Sistema

Para reproducir fielmente un escenario de auditoría forense de credenciales sin riesgos de alteración sobre bases de datos de producción, el equipo auditor configuró un laboratorio aislado bajo la distribución especializada **Kali Linux** en una máquina virtualizada.

1.1. Arquitectura de Cómputo y Backend OpenCL

Dado que el entorno virtualizado no disponía de transferencia directa de tarjeta gráfica dedicada (GPU *passthrough*), la suite criptográfica **Hashcat (v7.x)** se configuró para operar sobre los procesadores de la CPU virtual del hipervisor mediante la capa de abstracción OpenCL. Para garantizar la máxima tasa de cálculo y evitar cuellos de botella de memoria, se aplicaron dos modificadores obligatorios en todas las llamadas:

- `-D 1` (*Device Type: CPU*): Fuerza a Hashcat a seleccionar exclusivamente los núcleos de procesamiento central como aceleradores OpenCL.
- `-O` (*Optimized Kernels*): Activa la compilación de núcleos de aceleración optimizados para longitudes acotadas de contraseña, maximizando los ciclos por segundo.

2. Fase Previa: Normalización Criptográfica y Salting

El extracto remitido por la entidad financiera contenía los resúmenes codificados en **Base64** junto a campos numéricos de sal (*salt*).

2.1. Deducción del Algoritmo Criptográfico

Cada cadena en Base64 presentaba una longitud constante de 44 caracteres terminados en relleno (=). Al decodificar el contenido a formato binario puro se obtienen exactamente 32 bytes, lo que equivale a $32 \times 8 = 256$ bits. Esta longitud fija confirma inequívocamente el empleo de la función de dispersión criptográfica segura **SHA-256**.

Para preparar las entradas requeridas por Hashcat, se programó el script `b64_to_hex.py` (adjunto en el Anexo I), el cual realiza:

1. Decodificación de cada hash Base64 a su representación canónica en 64 caracteres hexadecimales.
2. Estructuración de ficheros en formato canónico `hash:salt`.
3. Generación de ficheros individuales (`hash_user1.txt` a `hash_user4.txt`) y global (`hashes_hashcat.txt`).

```
# Ejecucion del script de normalizacion
python3 b64_to_hex.py
```

2.2. Esquemas de Dispersión Evaluados (Prefijo vs. Sufijo)

En sistemas con salting existen dos disposiciones estructurales de concatenación:

- **Modo Sufijo (`-m 1410`):** $Hash = \text{SHA-256}(\text{Password} \,\|\, \text{Salt})$.
- **Modo Prefijo (`-m 1420`):** $Hash = \text{SHA-256}(\text{Salt} \,\|\, \text{Password})$.

Como se constata en la ejecución práctica, el sistema corporativo presentó divergencias en el orden de concatenación según la rutina de inserción de cada usuario.

3. Ejecución de Ataques Dirigidos y Recuperación de Credenciales

3.1. Usuario 1: Luis Toreman Gerto (Salt: `40`)

- **Pista declarada:** *«...mi contraseña es una palabra rara de la lengua inglesa».*
- **Vector:** Diccionario léxico directo (`-a 0`) sobre el corpus en inglés del sistema (`/usr/share/dict/words`).
- **Comando ejecutado:**

```
hashcat -m 1410 -a 0 -D 1 -O hash_user1.txt /usr/share/dict/words
```

- **Resultado:** RESOLUCIÓN EXITOSA **Contraseña:** `procrastination`
- **Evidencia:** `2153eacc93774e82650452fe8ac7149d7fb94df7a87f381813209bee9edce1d4:40:procrastination`
- **Análisis:** Longitud de 15 caracteres alfabéticos en minúsculas. Carece de mayúsculas, dígitos y símbolos.

3.2. Usuario 2: Ana Pearse Sanz (Salt: `51`)

- **Pista declarada:** *«...es de las contraseñas más comunes seguida de un punto y dos números».*
- **Vector:** Ataque híbrido (`-a 6`, *Wordlist + Mask*) con `rockyou.txt` y la máscara `.?d?d`.
- **Intento inicial fallido (Modo Sufijo `-m 1410`):**

```
hashcat -m 1410 -a 6 -D 1 -O hash_user2.txt /usr/share/wordlists/rockyou.txt .?d?d
```

  Diagnóstico: Concluyó en estado **Exhausted** (AGOTADO / CAMBIO DE MODO) al no haber sido concatenada la sal como sufijo.
- **Corrección técnica definitiva (Modo Prefijo `-m 1420`):**

```
hashcat -m 1420 -a 6 -D 1 -O hash_user2.txt /usr/share/wordlists/rockyou.txt .?d?d
```

- **Resultado:** RESOLUCIÓN EXITOSA **Contraseña:** `12345678.22`
- **Evidencia:** `ba4f0a10cff5df84345dbc4bf6de4cec5f1f8fdf0b9265680c8b759ea11069d5:51:12345678.22`
- **Análisis:** Secuencia trivial `12345678` combinada con delimitador y sufijo `22`. Carece de letras.

3.3. Usuario 3: Pedro Marteis Poncio (Salt: `18`)

- **Pista declarada:** *«...es la fecha de nacimiento de uno de sus hijos».*
- **Vector:** Fuerza bruta mediante máscara numérica de 8 posiciones (`-a 3 ?d?d?d?d?d?d?d?d`, formato $DDMMAAAA$).
- **Comando ejecutado:**

```
hashcat -m 1410 -a 3 -D 1 -O hash_user3.txt ?d?d?d?d?d?d?d?d
```

- **Resultado:** RESOLUCIÓN EXITOSA **Contraseña:** `29021984`
- **Evidencia:** `14fc0f8507797e252f7a99721978df03590d4c024e97ecd6c0d797b5327f73be:18:29021984`
- **Análisis:** Fecha real correspondiente al 29 de febrero de 1984. Longitud de 8 caracteres enteramente numéricos.

3.4. Usuario 4: Arturo Ruiz Gasca (Salt: `31`)

- **Pista declarada:** *«...es la combinación de mi nombre y apellidos con el símbolo *».*
- **Modelado sintético:** En lugar de fuerza bruta genérica, se generó un diccionario sintético acotado (`diccionario_arturo.txt`) mediante el script `generador_arturo.py` permutando sus identidades con asteriscos.
- **Comando ejecutado:**

```
hashcat -m 1410 -a 0 -D 1 -O hash_user4.txt diccionario_arturo.txt
```

- **Resultado:** RESOLUCIÓN EXITOSA **Contraseña:** `ruiz*arturo*gasca`
- **Evidencia:** `4baf587b8197529730b00a62be7f91278de1f8f52faca43c2d6baa4a1b70b7f5:31:ruiz*arturo*gasca`
- **Análisis:** Cadena de 17 caracteres en minúsculas unidas por asteriscos. Carece de letras mayúsculas y dígitos.

4. Consolidación Forense de Evidencias Digitales

Tras descifrar las cuatro credenciales, se volcaron los resultados del fichero de estado (*potfile*) de Hashcat hacia los registros de entrega mediante la opción `--show`:

```
cat solucion_1.txt solucion_2.txt solucion_3.txt solucion_4.txt
cat soluciones.txt
```

Usuario	Salt	Modo	Contraseña Recuperada	Vector / Diccionario
Luis Toreman Gerto	40	`-m 1410`	`procrastination`	Corpus léxico inglés (`words`)
Ana Pearse Sanz	51	`-m 1420`	`12345678.22`	Híbrido: `rockyou.txt` + máscara `.?d?d`
Pedro Marteis Poncio	18	`-m 1410`	`29021984`	Máscara de fechas: `?d?d?d?d?d?d?d?d`
Arturo Ruiz Gasca	31	`-m 1410`	`ruiz*arturo*gasca`	Diccionario sintético (`generador_arturo.py`)

Cuadro 1: Parámetros de resolución y credenciales descifradas.

5. Lógica Algorítmica y Pseudocódigo de Scripts Auxiliares

Para mayor claridad y legibilidad, se resume la lógica procedimental de los scripts auxiliares programados. El código fuente completo, ejecutable y comentado se encuentra adjunto en los archivos `b64_to_hex.py` (Anexo I) y `generador_arturo.py` (Anexo II).

5.1. Algoritmo de Decodificación y Normalización (`b64_to_hex.py`)

```
ALGORITMO NormalizarHashesBase64:
  Entrada: Lista de usuarios con campos (Nombre, HashBase64, Salt, Pista)
  Para cada usuario en la lista:
    1. Decodificar bytes_crudos = Base64_Decode(usuario.HashBase64)
    2. Convertir digest_hex = Hexadecimal_Encode(bytes_crudos)
    3. Validar longitud: Verificar que Longitud(digest_hex) == 64 (256 bits -> SHA-256)
    4. Estructurar registro con formato Hashcat: "digest_hex:Salt"
  Guardar registros en archivo estructurado JSON ("hashes_cai1.json")
  Exportar lineas a formato Hashcat general ("hashes_hashcat.txt")
  Generar ficheros individuales para ataques dirigidos ("hash_user1.txt" ... "hash_user4.txt")
FIN ALGORITMO
```

5.2. Algoritmo Generador de Diccionario Sintético (`generador_arturo.py`)

```
ALGORITMO GenerarDiccionarioArturo:
  Definir componentes_nominales = ["arturo", "ruiz", "gasca"]
  Definir conjunto_candidatos = ConjuntoVacio()

  // 1. Permutaciones de orden 3 con delimitador '*' y casing (minusculas, mayusculas, capital)
  Para cada permutacion (w0, w1, w2) en Permutaciones(componentes_nominales, 3):
    Para cada variante de caja (minuscula / mayuscula / capital) de w0, w1, w2:
      Insertar en candidatos: "w0*w1*w2", "w0*w1w2", "w0w1*w2", "*w0w1w2", "w0w1w2*"

  // 2. Permutaciones de orden 2 y palabras individuales delimitadas
  Para cada permutacion (w0, w1) en Permutaciones(componentes_nominales, 2):
    Insertar en candidatos combinaciones binaras con '*': "w0*w1", "*w0*w1", "w0*w1*"

  Para cada componente individual:
    Insertar sufijos y prefijos con asterisco: "w*", "*w", "*w*"

  Exportar candidatos ordenados a fichero de texto ("diccionario_arturo.txt")
FIN ALGORITMO
```

Memoria técnica complementaria elaborada por SecurTech Advisory para el entregable CAI 1.